

An effective artificial bee colony algorithm for the flexible job-shop scheduling problem

Ling Wang · Gang Zhou · Ye Xu · Shengyao Wang · Min Liu

Received: 16 January 2011 / Accepted: 23 August 2011 / Published online: 2 September 2011
© Springer-Verlag London Limited 2011

Abstract An effective artificial bee colony (ABC) algorithm is proposed in this paper for solving the flexible job-shop scheduling problem with the criterion to minimize the maximum completion time (makespan). The ABC algorithm stresses the balance between global exploration and local exploitation. First, multiple strategies are utilized in a combination to generate the initial solutions with certain quality and diversity as the food sources. Second, crossover and mutation operators are well designed for machine assignment and operation sequence to generate the new neighbor food sources for the employed bees. Third, a local search strategy based on critical path is proposed and embedded in the searching framework to enhance the local intensification capability for the onlooker bees. Meanwhile, an updating mechanism of population by generating the scout bees with the initialing strategy is proposed to enrich the searching behavior and avoid the premature convergence of the algorithm. In addition, a well-designed left-shift decoding scheme is employed to transform a solution to an active schedule. Numerical simulation results based on well-known benchmarks and comparisons with some existing algorithms demonstrate the effectiveness of the proposed ABC algorithm.

Keywords Flexible job-shop scheduling problem · Makespan · Artificial bee colony algorithm · Machine assignment · Operation sequence · Critical path

1 Introduction

The flexible job-shop scheduling problem (FJSP) is a generalization of the classical job-shop scheduling problem (JSP) and the parallel machine environment with a strong engineering background, such as in textile industry, mechanical manufacturing systems, chemical engineering industry, automobile assembly process, semiconductor manufacturing process, and transportation systems. The FJSP extends the classical job-shop scheduling by allowing an operation to be processed on any machine from a given set along different routes. The FJSP consists of two sub-problems, i.e., routing and scheduling. The routing sub-problem is to assign each operation to a machine among a set of given machines, whereas the scheduling sub-problem is to sequence the assigned operations on all machines to obtain a feasible schedule with a satisfactory objective value. So, for the FJSP, it should assign each operation to a machine appropriately and sequence the operations reasonably to make the scheduling objective optimal. In addition to the common complexities of the JSP, the FJSP is of a larger complexity in determining the assignment of operations to machines as well as the sequence of all operators. It has been proved that the FJSP is NP-hard. Hence, the study of the FJSP in theory, methodology, and applications has significant importance in both academic field and engineering field.

So far, many solution algorithms have been proposed for the FJSP. In the first work to address the FJSP by Bruker and Schlie [1], a polynomial algorithm was proposed to solve the FJSP with two jobs, in which each operation has the same processing time on different machines. To solve the realistic problems with more than two jobs, two types of techniques have been developed: hierarchical approaches and integrated approaches. The hierarchical approaches

L. Wang (✉) · G. Zhou · Y. Xu · S. Wang · M. Liu
Tsinghua National Laboratory for Information Science
and Technology (TNList), Department of Automation,
Tsinghua University,
Beijing 100084, China
e-mail: wangling@mail.tsinghua.edu.cn

treat the routing and scheduling independently, whereas the integrated approaches process the two sub-problems jointly.

Based on the idea of decomposing the original problem into sub-problems, hierarchical approaches are natural for the FJSP due to the fact that the routing and the scheduling sub-problems of the FJSP can be separated correspondingly. Brandimarte [2] proposed a hierarchical approach based on decomposition to solve the FJSP, in which the routing sub-problem was solved by some existing dispatching rules, and then the sequencing sub-problem was solved using tabu search (TS). A similar hierarchical approach was proposed by Paulli for the flexible manufacturing system [3].

Due to the complexity of the FJSP, meta-heuristics and evolutionary algorithms have been widely applied to the FJSP. In [4, 5], a localization approach was developed to solve the assignment problem, and an evolutionary approach controlled by the assignment model was proposed for the FJSP. Xia and Wu [6] proposed a hierarchical approach by using particle swarm optimization (PSO) to assign operations to machines and by using simulated annealing (SA) to schedule operations on each machine for solving the multi-objective FJSP. In [7], an architecture called learnable genetic architecture (LEGA) was developed for learning and evolving solutions for the FJSP. In [8], a TS algorithm with two heuristics was proposed by Saidi-mehrabad and Fattahi to solve the FJSP with sequence-dependent setups, and in [9], SA and TS algorithms were presented for solving the FJSP. In [10], genetic algorithm (GA) was hybridized with variable neighborhood search to solve the multi-objective FJSP. In [11], a hierarchical algorithm hybridizing the machine selection module and the operation scheduling module was proposed for solving the FJSP with parallel machines and reentrant process. In [12], a variable neighborhood genetic algorithm was presented for the FJSP with makespan criterion. In [13], a multistage operation-based GA was proposed to deal with the problem from the point of view of dynamic programming. Recently, Xing et al. proposed a knowledge-based ant colony optimization algorithm (KBACO) for solving the FJSP [14], and Yazdani et al. developed a parallel variable neighborhood search (PVNS) algorithm based on six neighborhood structures [15]. Moradi et al. proposed an efficient architecture for scheduling the FJSP with machine availability constraints [16]. Defersha and Chen presented a parallel GA for the FJSP with sequence-dependent setups [17]. Zhang et al. proposed an effective GA for solving the FJSP to minimize makespan time [18]. Li et al. developed a hybrid TS with an efficient neighborhood structure for the FJSP [19].

Compared to the hierarchical approaches, integrated approaches are more difficult to solve but generally can achieve better results. Dauzere-Peres and Paulli [20] proposed a TS algorithm based on the integrated approach and defined a new neighborhood structure for the problem,

in which there was no distinction between reassigning and re-sequencing an operation. Mastrolilli and Gambardella [21] developed two neighborhood functions and improved the TS algorithm in [20] with better performances than any other existing meta-heuristics in terms of computation time and solution quality. Baykasoglu [22] proposed an approach based on linguistics, SA, and a dispatching rule, in which the FJSP was defined as alternative process plans. Chan et al. [23] used an integrated approach based on GA to solve the FJSP under resource constraints. Pezzella et al. [24] introduced a GA integrating different strategies for generating the initial population, selecting individuals for reproduction and reproducing new individuals to solve the FJSP. Tay and Ho [25] developed an integrated approach based on genetic programming for solving the multi-objective FJSP to minimize makespan, mean tardiness, and mean flow times. Most recently, Moslehi and Mahnam [26] proposed a Pareto approach using PSO and local search for the multi-objective FJSP, and the computational results proved the efficiency of the algorithm.

As a relatively new member of swarm intelligence, artificial bee colony (ABC) algorithm [27] is based on collective behavior of self-organized systems. So far, ABC algorithm has received interest from researchers in a variety of fields. The ABC algorithm was first proposed to optimize multi-variable and multi-modal continuous functions [27]. Many comparative studies showed that the performance of the ABC algorithm was competitive to other population-based algorithms with the advantage of employing fewer control parameters in the continuous space [28–31]. Compared with the applications of ABC to the continuous optimization problems, the applications of ABC to the combinatorial optimization problems are considerably limited. Baykasoglu et al. [32] made a comparative study of ABC and developed an effective artificial bee colony algorithm to solve the generalized assignment problem. Recently, Wong et al. [33] developed a novel bee colony optimization with local search to solve traveling salesman problem, and Pan et al. [34] proposed a discrete artificial bee colony algorithm based on a self-adaptive strategy to solve the lot-streaming flow shop scheduling problem with the criterion of total weighted earliness and tardiness penalties.

To the best of our knowledge, there is no published work to solve the FJSP by using the ABC algorithm. In this paper, we will propose an effective ABC algorithm for solving the FJSP with a criterion of makespan. The proposed ABC algorithm is well designed to stress the balance between global exploration and local exploitation. With the evolutionary searching mechanism of ABC characterized by individual improvement, cooperation, and competition of the population, the algorithm can perform global exploration efficiently. With the novel local search based on critical path, the algorithm can perform local

exploitation effectively. First, an efficient decoding rule based on left-shift operation is employed to transform a solution to an active schedule. Second, an efficient population initialization based on the combination of multiple strategies is incorporated into the random initialization of ABC algorithm to generate an initial population with certain quality and diversity. Meanwhile, crossover and mutation operators are well designed to generate new neighbor food sources in the employed bee phase; a local search strategy based on critical path is developed to improve the convergence speed in the onlooker bee phase; a simple but effective updating strategy is applied in the scout bee phase to enrich the population and to avoid premature convergence. Simulation results based on well-known benchmarks and comparisons with some existing algorithms demonstrate the effectiveness of the proposed ABC for solving the FJSP.

The remainder of the paper is organized as follows: In Section 2, the FJSP is described and formulated. In Section 3, the traditional ABC is introduced briefly. In Section 4, the framework of ABC algorithm for the FJSP is proposed after presenting the solution representation, decoding scheme, population initialization, crossover and mutation in the employed bee phase, local search in the onlooker bee phase, and updating strategy in the scout bee phase in detail. Simulation results on well-known benchmarks and the comparisons to some existing algorithms are presented and analyzed in Section 5. Finally, we end the paper with some conclusions and future work in Section 6.

2 Problem formulation

The FJSP is commonly defined as follows: There are a set of n jobs to be processed on m machines. The set of machines is denoted as $M = \{M_1, M_2, \dots, M_m\}$. Each job i consists of a sequence of n_i operations $O_{i,j}, j = 1, 2, \dots, n_i$. The processing time $p_{i,j,k}$ of $O_{i,j}$ performed on machine k is given. Each routing has to be determined to complete a job. The execution of $O_{i,j}$ requires one machine out of a set of given machines $M_{i,j} \subseteq M$. Let $S_{i,j}$ and $C_{i,j}$ be the starting time and completion time of operation $O_{i,j}$. Let $S_{k,r}^m$ be the starting time of $O_{i,j}$ performed on machine k in priority r . Let q_k be the number of operations assigned to machine k . Let $C_{\max}$ be the maximum completion time. The FJSP is to determine both the assignment of machines and the sequence of operations on all the machines to minimize the scheduling objective, e.g., $C_{\max}$.

In addition, we suppose that (1) setting up times of machines and move times between operations are negligible, (2) machines are independent from each other and jobs are independent from each other as well, (3) at one time a machine can execute at most one operation and it becomes available to

other operations only if the previous operation is completed, and (4) there is no precedence constraint among the operations of different jobs.

Mathematically, the FJSP can be formulated as the following mixed integer program problem [19]:

$$\text{Minimize } C_{\max} \quad (1)$$

Subject to:

$$S_{i,j} + \sum_{k \in M_{i,j}} (p_{i,j,k} \cdot x_{i,j,k}) \leq S_{i,j+1}, i = 1, 2, \dots, n; j = 1, 2, \dots, n_i - 1 \quad (2)$$

$$\sum_{k \in M_{i,j}} x_{i,j,k} = 1, i = 1, 2, \dots, n; j = 1, 2, \dots, n_i; k = 1, 2, \dots, m \quad (3)$$

$$\sum_r y_{i,j,k,r} = x_{i,j,k}, i = 1, 2, \dots, n; j = 1, 2, \dots, n_i; k = 1, 2, \dots, m; r = 1, 2, \dots, q_k \quad (4)$$

$$S_{k,r}^m + p_{i,j,k} \cdot y_{i,j,k,r} \leq S_{k,(r+1)}^m, i = 1, 2, \dots, n; j = 1, 2, \dots, n_i; k = 1, 2, \dots, m; r = 1, 2, \dots, q_k - 1 \quad (5)$$

$$S_{k,r}^m + (1 - y_{i,j,k,r}) \cdot L \geq S_{i,j}, i = 1, 2, \dots, n; j = 1, 2, \dots, n_i; k = 1, 2, \dots, m; r = 1, 2, \dots, q_k \quad (6)$$

$$S_{k,r}^m \leq S_{i,j} + (1 - y_{i,j,k,r}) \cdot L, i = 1, 2, \dots, n; j = 1, 2, \dots, n_i; k = 1, 2, \dots, m; r = 1, 2, \dots, q_k \quad (7)$$

$$x_{i,j,k} = \begin{cases} 1, & \text{if machine } k \text{ is selected for operation } O_{i,j} \\ 0, & \text{otherwise.} \end{cases} \quad (8)$$

$$y_{i,j,k,r} = \begin{cases} 1, & \text{if operation } O_{i,j} \text{ is performed on machine } k \text{ in priority } r \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

where Eq. 1 is the objective function, Eq. 2 ensures that the operations belonging to the same job satisfy the precedence constraints, Eq. 3 ensures that each operation is assigned to only one machine from its candidate machine set, Eq. 4 guarantees that each operation should be processed only at one priority on one machine, Eq. 5 ensures that each machine processes one operation at a time, and Eqs. 6 and 7 guarantee that each operation should be started only after

Fig. 1 The procedure of the ABC algorithm

Step 1: Initialize parameters, including the number of food sources or the number of the employed bees (n_{eb}), number of onlooker bees (n_{ob}), number of scout bees (n_{sb}).

Step 2: Initialize population of food sources with random solutions.

Step 3: Calculate the objective value $f(s_i)$ of each food source s_i , and then determine the best food resource $gbest$.

Step 4 (employed bee phase):

Step 4.1: For every employed bee $u_i, i=1,2,\dots,n_{eb}$, generate the new food source s_i^1 from s_i ;

Step 4.2 (onlooker bee phase):

Step 4.2.1: Calculate the probability of selecting food source s_i^1 according to Eq. (10);

Step 4.2.2: Calculate the number of onlooker bees $N_i = p_i \cdot n_{ob}$ to be sent to the food source s_i^1 ;

Step 4.2.3: For every onlooker bee $v_j, j=1,2,\dots,N_i$, generate the N_i new food sources $s_{i,j}^2$ from s_i^1 . And let s_i^{best} be the best of all the $s_{i,j}^2$. If $f(s_i^{best}) \leq f(s_i)$ then $s_i = s_i^{best}$;

Step 4.3: For $i=1,2,\dots,n_{eb}$: if $f(s_i) \leq f(gbest)$, then $gbest = s_i$;

Step 5 (scout bee phase):

Step 5.1: Initialize scout bees with random solutions and update $gbest$ if possible;

Step 5.2: Determine the worst n_{sb} employed bees and replace them with the scout bees if the scout bees are better.

Step 6: If a stopping criterion is met, then output $gbest$ and its objective value; otherwise, go back to Step 4.

the previous operation of the same machine and the previous operation of the same job are both completed and L is a number large enough.

3 The artificial bee colony algorithm

Inspired by the intelligent foraging behaviors of honeybee swarm, the artificial bee colony algorithm is a new population-based meta-heuristic approach [27–31]. In the ABC algorithm, there are three kinds of foraging bees, that is, employed bees, onlooker bees, and scout bees. Employed bees are those bees that are exploiting a food source currently, onlooker bees are those bees that are waiting in the hive for the information to be shared by the employed bees about their food sources, and scout bees are those bees that are currently searching for new food sources in the vicinity of the hive.

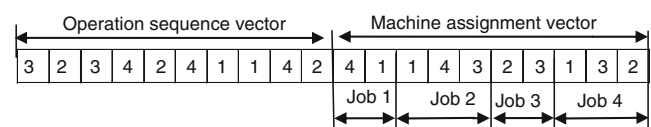
The work process of the bees can be described as follows: The employed bees bring loads of nectar from the

food sources to the hive and then share the food source information with onlooker bees by dancing in a common area in the hive called dance area. The duration of a dance is proportional to the nectar content of the food source currently being exploited by the dancing bee. Onlooker bees need to watch numerous dances before choosing a food source, which tend to choose a food source according to the probability proportional to the quality of that food source. Therefore, the good food sources tend to attract more bees than the bad ones. A scout or onlooker bee may change into an employed bee when it finds a better source. An employed bees associated with a food source may become a scout or onlooker bee when the food source is exploited fully. Employed bees and scout bees can be visualized as performing exploration whereas onlooker bees can be visualized as performing exploitation.

In the ABC algorithm, each food source represents a possible solution to the problem under consideration, and the nectar amount of a food source represents the quality of the solution. The ABC algorithm assumes that there is only one employed bee for every food source, i.e., the number of food sources is the same as the number of employed bees. The employed bee of an abandoned food source becomes a scout bee, and as soon as it finds a new food source, it becomes an employed bee again. The ABC algorithm is an iterative algorithm. It starts by associating all employed bees with randomly generated food sources (solution). Then every employed bee moves to a new food source in the neighborhood of its currently associated food source and evaluates its nectar amount

Table 1 Processing time table

Operations	Machines			
	M_1	M_2	M_3	M_4
$O_{1,1}$	4	7	6	5
$O_{1,2}$	2	6	∞	5
$O_{2,1}$	4	5	7	∞
$O_{2,2}$	5	∞	6	3
$O_{2,3}$	∞	5	4	7
$O_{3,1}$	5	3	∞	6
$O_{3,2}$	∞	∞	4	∞
$O_{4,1}$	2	4	5	∞
$O_{4,2}$	∞	4	2	∞
$O_{4,3}$	5	4	6	3

**Fig. 2** Illustration of the representation of a feasible solution

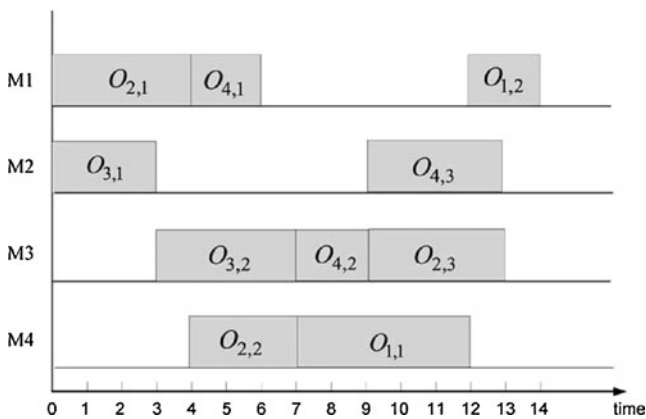


Fig. 3 Gantt chart of the solution shown in Fig. 2

(objective value) during iterations. When the employed bees complete the process, they share the nectar information of the food sources with the onlooker bees, and the number of onlooker bees to be sent to the food source found by the employed bee is proportional to the nectar amount of that food source. The probability p_i of selecting a food source s_i is determined by the following expression:

$$p_i = \frac{[f(s_i)]^{-1}}{\sum_{j=1}^{n_{fs}} [f(s_j)]^{-1}} \quad (10)$$

where $f(s_i)$ is the objective value of the solution represented by the food source s_i and n_{fs} is the total number of food sources. Clearly, good food sources will attract more onlookers than the bad ones. The procedure of the ABC algorithm [32] can be illustrated as Fig. 1.

4 ABC for FJSP

4.1 Solution representation

The solution of the FJSP is a combination of machine assignment and operation scheduling decisions. So, a

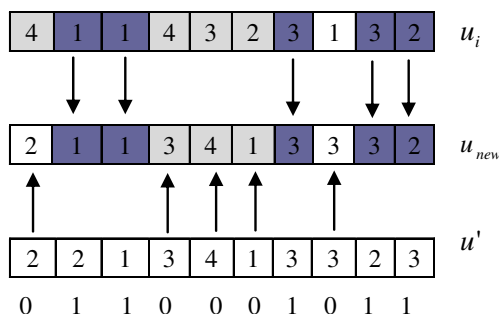


Fig. 4 Illustration of the procedure of uniform crossover

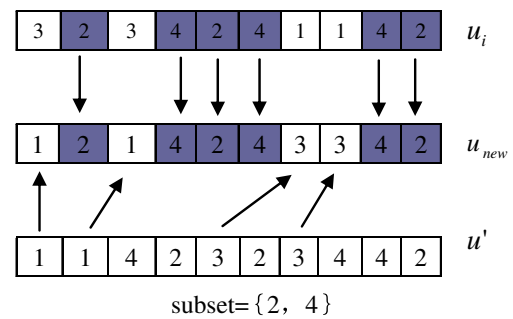


Fig. 5 Illustration of the procedure of MPOX

solution can be expressed by the assignment of operations on machines and the processing sequence of operations on the machines. According to the representation [35], we integrate the operation sequence vector and machine assignment vector as the representation of a feasible solution.

To explain the representation, we provide an example by considering a problem with four jobs and four machines as shown in Table 1. And Fig. 2 illustrates the representation of a feasible solution.

In Fig. 2, the first part is used to sequence the operations while the second part is used to assign a suitable machine for each operation. For the operation sequence vector, the number of genes equals to the total number of operations. The operations of each job are denoted by the corresponding job number, and the k th occurrence of a job number refers to the k th operation in the sequence of this job. For the machine assignment vector, each number represents the machine assigned for each operation successively. Then the representation has the following operation sequence and machine assignment: $(O_{3,1}, M_2)$, $(O_{2,1}, M_1)$, $(O_{3,2}, M_3)$, $(O_{4,1}, M_1)$, $(O_{2,2}, M_4)$, $(O_{4,2}, M_3)$, $(O_{1,1}, M_4)$, $(O_{1,2}, M_1)$, $(O_{4,3}, M_2)$, $(O_{2,3}, M_3)$. The Gantt chart of this solution is shown in Fig. 3.

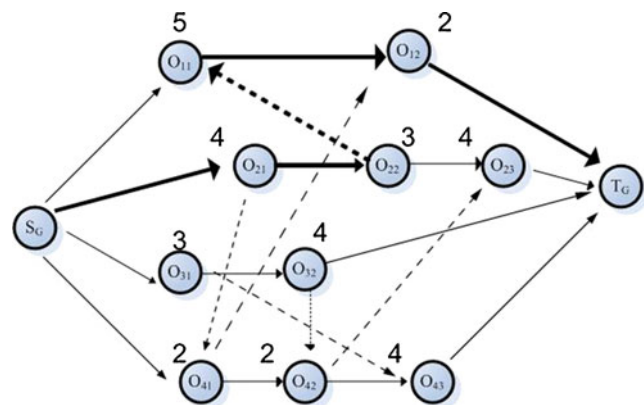


Fig. 6 Illustration of a disjunctive graph

Input: solution s_i^j found by an employed bee $u_i, i=1,2,\dots, n_{eb}$
Output: improved solution $s_{i,new}, i=1,2,\dots, n_{eb}$
Begin
 For $j=1$ to N_i
 Generate $s_{i,new}$ by moving all critical operations of s_i^j ;
 Let $s_i^j = s_{i,new}$;
 End For
 Output $s_{i,new}$
End

Fig. 7 Local search for onlooker bees

4.2 Decoding scheme

Decoding is the process to transfer the representation to the schedule. Due to the precedence constraint among operations of the same job, idle time may exist between operations on a machine. Considering the criterion of makespan, we employ the left-shift scheme to shift the

operations to the left as compact as possible. Given a time interval $[t_k^S, t_k^E]$ beginning from t_k^S and ending at t_k^E on the machine k to allocate an operation O_{ij} , O_{ij} can be started only after its immediate job predecessor $O_{i,j-1}$ is completed so that the starting time of O_{ij} can be described as follows. If the operation has no predecessor, $C_{i,j-1}$ is replaced with “0.”

$$S_{ij} = \max\{t_k^S, C_{i,j-1}\} \quad (11)$$

When operation O_{ij} is assigned on machine k , we examine the idle time intervals between operations that have already been scheduled on the machine from left to right to find the earliest available time S_{ij} . If there is enough time span from S_{ij} until t_k^E to complete O_{ij} , i.e., $\max\{t_k^S, C_{i,j-1}\} + p_{ij,k} \leq t_k^E$, then time interval $[t_k^S, t_k^E]$ is

Fig. 8 The framework of ABC for FJSP

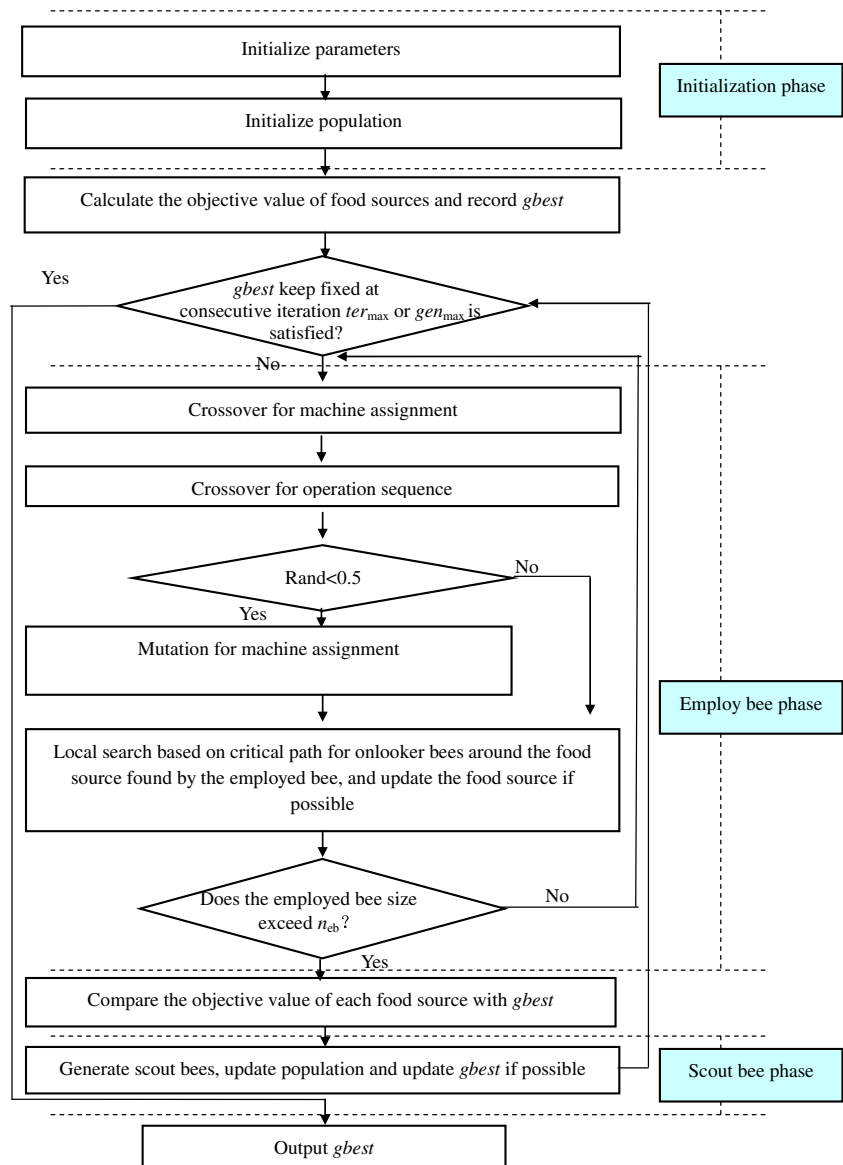


Table 2 Results of the five Kacem instances

Instance	$n \times m$	C^*	AL + CGA	GEN ACE	PSO + TS	PVNS		KBACO		TSPCB		Proposed ABC			
						BEST	AVG	BEST	AVG	BEST	AVG	BEST	AVG	SD	TAVG (s)
Case 1	4×5	11	16	11	N/A	N/A	N/A	11	11.00	11	11.00	11	11.00	0.00	0.04
Case 2	8×8	14	15	N/A	15	14	14.00	14	14.30	14	14.20	14	14.00	0.00	0.29
Case 3	10×7	11	15	12	N/A	N/A	N/A	11	11.00	11	11.00	11	11.00	0.00	1.58
Case 4	10×10	7	7	7	7	7	7.00	7	7.40	7	7.10	7	7.00	0.00	0.45
Case 5	15×10	11	23	12	12	12	12.00	11	11.30	11	11.70	11	11.00	0.00	5.39

The bold value means the best result

N/A the result is non-available

available for O_{ij} . That is, O_{ij} can be left-shifted. We use such a left-shift decoding scheme to allocate each operation on its assigned machine from left to right following the operation sequence vector of the representation. In this way, a representation is decoded to a detailed schedule.

4.3 Population initialization

To guarantee an initial population with a certain quality and diversity, some different strategies are utilized in a hybrid way to generate the initial solutions as the food sources. The initialization process includes machine assignment phase and operation sequence phase.

To generate initial machine assignments, the following three rules are applied: (1) random rule. Randomly select a machine from the candidate machines set for each operation and then place it at the position in the machine assignment vector. This rule is simple to implement whereas it may cause the machine workload unbalanced; (2) local minimum processing time rule. Please refer [19] for the detailed implementation. The main advantage of this rule is attributed to its consideration of the workload among the set of machines waiting for processing the same job; and (3) global minimum processing time rule. Please refer [24] for the detailed implementation. The main advantage of this rule is attributed to its consideration of the global workload among all machines. In our algorithm, the above three rules are used in a hybrid way to generate initial population of machine assignments. In particular, the initial machine assignments of 60% solutions in the population are generated by global minimum processing time rule, 30% solutions by local minimum processing time rule, and 10% solutions by random rule.

Once the machines are assigned to all the operations, all the operations will be sequenced. A schedule is feasible only all the precedence constraints among operations of the same job are not violated. We also apply three rules to generate initial operation sequences: (1) random rule.

Randomly generate the sequence of the operations on each machine. Such a rule is simple to implement whereas it may produce large idle time interval; (2) most time remaining rule [2]. Sequence the jobs in the order of non-increasing remaining time, that is, the job with the most remaining time will be selected first; and (3) most number of operations remaining rule [24]. The job with most remaining operations unprocessed has a high priority to be selected. In our algorithm, the above three rules are used in a hybrid way to generate initial population of operation sequence. In particular, the initial operation sequences 20% solutions in the population are generated by random rule, 40% solution by most time remaining rule, and 40% solutions by most number of operations remaining rule.

4.4 Crossover and mutation in the employed bee phase

In our ABC algorithm, employed bees perform global exploration with multiple different neighborhoods for promising food sources over the entire region. Since the FJSP consists of machine assignment and operation sequence, we well design the crossover and mutation operators to evolve the machine assignment and operation sequence.

4.5 Crossover for machine assignment

To evolve the machine assignment, two crossover operators are applied with equal probability, i.e., the two-point crossover and uniform crossover. These crossover operators only change the machine assignment but not change the operation sequence. To the two feasible parents, the offspring generated by these crossover operators is still feasible. Please refer [36] for the implementation of two-point crossover. The procedure of uniform crossover is described as follows:

Step 1. For every machine assignment vector of employed bee solution u_i , $i=1,2,\dots,n_{eb}$, randomly select three

Table 4 Relative percentage improvement (percent) of ABC to other algorithms (BRdata instances)

Problem	$n \times m$	TS	GA	LEGA	PVNS	KBACO	TSPCB
Mk1	10×6	+4.76	0.00	0.00	0.00	−2.56	0.00
Mk2	10×6	+18.75	0.00	+10.34	0.00	+10.34	0.00
Mk3	15×8	+3.32	0.00	N/A	0.00	0.00	0.00
Mk4	15×8	+25.93	0.00	+10.45	0.00	+7.69	+3.23
Mk5	15×4	+7.53	+0.58	+2.27	+0.58	+0.58	0.00
Mk6	10×15	+30.23	+4.76	+10.45	0.00	+10.45	+7.69
Mk7	20×5	+11.46	0.00	+5.44	+1.42	+3.47	+0.71
Mk8	20×10	0.00	0.00	0.00	0.00	0.00	0.00
Mk9	20×10	+16.80	+1.29	+4.06	0.00	+1.29	+0.97
Mk10	20×15	+29.73	+1.89	+9.17	0.00	+9.17	+2.80
Average		+14.85	+0.85	+5.22	+0.20	+4.04	+1.54

4.6 Local search based on critical path in the onlooker bee phase

In addition to the crossover and mutation operators for the employed bees, a local search based on critical path is designed for the onlooker bees to enhance the local exploitation around the solutions found by the employed bees.

4.6.1 Critical path theory

The feasible schedule of the FJSP can be represented with a disjunctive graph $G=(N,A,E)$ with node set N , conjunctive arc set A , and disjunctive arc set E . The conjunctive arcs

denoted by real lines correspond to precedence relations of the same job, and the disjunctive arcs denoted by dashed lines correspond to precedence of operations performed on the same machine. The critical path is the longest path in G , and any operation on the critical path is called a critical operation. An example is showed in Fig. 6.

In Fig. 6, S_G and T_G are dummy starting and terminating nodes, respectively. The processing time of each operation is labeled on the node. Denote S_{ij}^E as the earliest starting time of operation O_{ij} and S_{ij}^L as the latest starting time without delaying the makespan. Thus, the earliest completion time of operation O_{ij} is $C_{ij}^E = S_{ij}^E + p_{ij,k}$, and the latest completion time is $C_{ij}^L = S_{ij}^L + p_{ij,k}$.

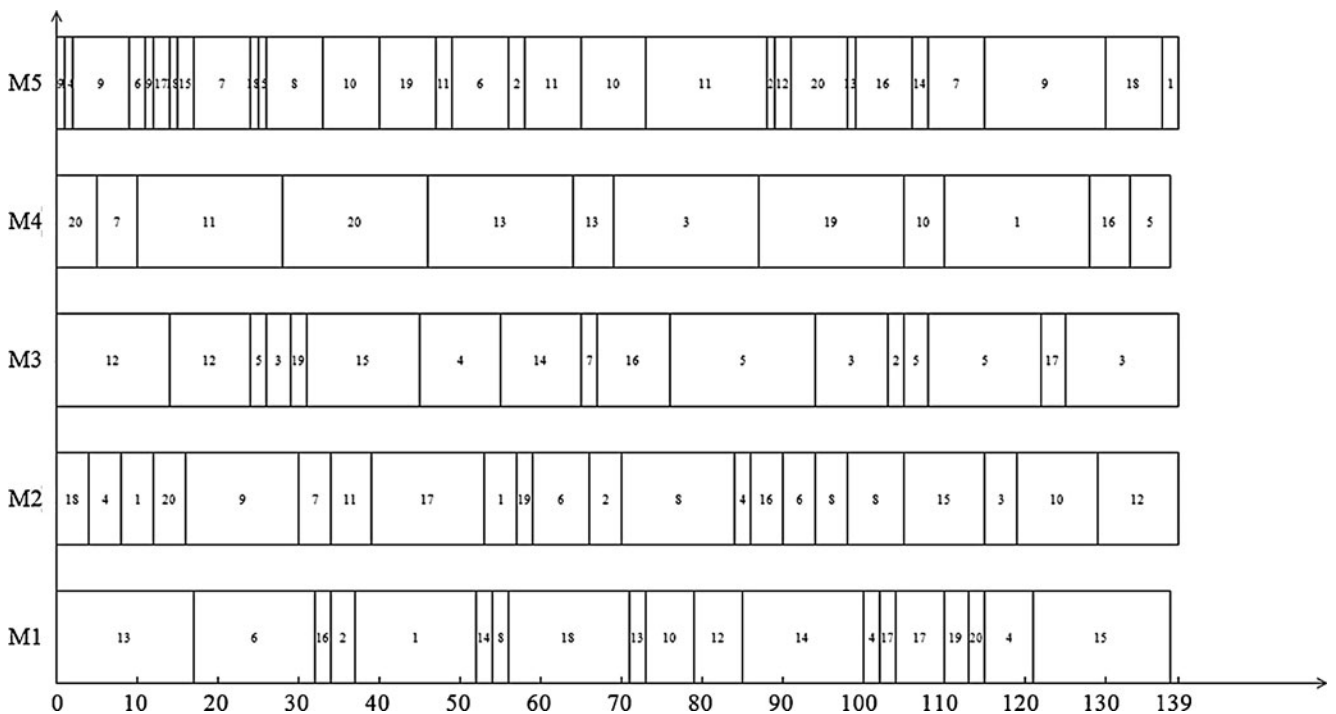
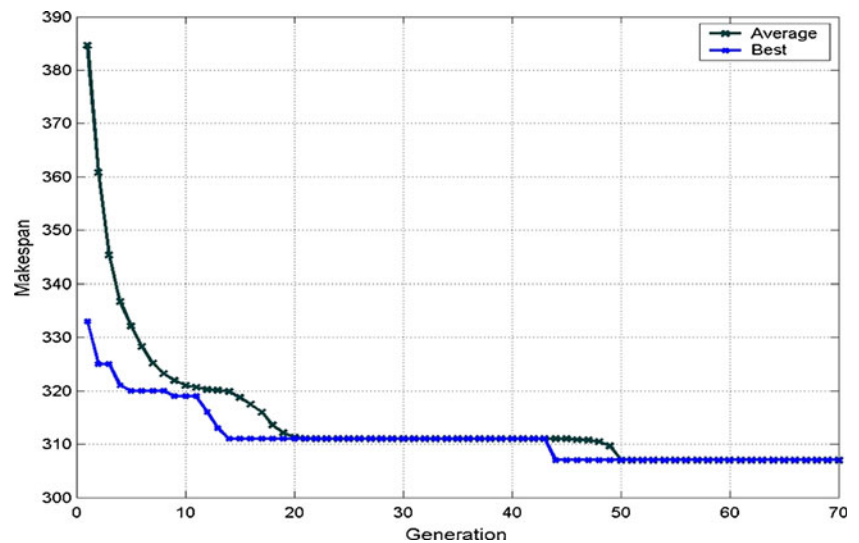
**Fig. 9** The best solution of Mk7 obtained by ABC (makespan=139)

Fig. 10 Convergence curves in solving MK9 by ABC



Since the makespan of a schedule is defined by the length of its critical path, any delay in the start of the critical operation will delay the completion of the schedule so as to worsen the makespan. So, if an operation belongs to the critical path, its starting time cannot be delayed and its earliest starting time must be equal to its latest starting time. Let PM_{ij}^k be the operation processed on machine k right before the operation O_{ij} and SM_{ij}^k be the operation processed on machine k right after O_{ij} . Let $PJ_{ij}=O_{i,j-1}$ be the operation of job i that precedes O_{ij} and $SJ_{ij}=O_{i,j+1}$ be the operation of job i that follows O_{ij} . The starting time of S_G is $S^E(S_G)=0$, and the earliest starting time of each node is $S_{ij}^E = \max\{C^E(PJ_{ij}), C^E(PM_{ij}^k)\}$. If a node has no job predecessor, then $C^E(PJ_{ij})=0$; if it has no machine predecessor, then $C^E(PM_{ij}^k) = 0$. To find the latest starting time of each node, a process beginning with the terminating node T_G called “work back” is applied. The latest completion time of T_G is equal to the makespan of the schedule, i.e., $C^L(T_G) = C_M(G)$, and the latest completion time of each node is $C_{ij}^L = \min\{S^L(SJ_{ij}), S^L(SM_{ij}^k)\}$. If a node has no job successor or machine successor, the corresponding completion time is replaced with an infinity number. After finding all the critical operations, the critical path will be determined. For example, the path marked with broad-brush lines in Fig. 6 denotes the critical path.

4.6.2 Local search based on moving critical operations

Since the makespan is no shorter than any possible path in the disjunctive graph, the makespan may be improved only by moving the critical operations. Let O_l ($l=1,2,\dots,N_c$) be the critical operation to be moved, where N_c is the total number of critical operations of a solution in G . Moving O_l is to delete it from its current position to get a new graph

G^- and then to insert it at another feasible position. Obviously, the makespan of G^- is no larger than the makespan of G . To find another feasible position in G^- to reallocate O_l to generate an improved solution, $C_M(G)$ is first taken as the “required” makespan of G^- when calculating the latest starting and completion time for each operation that remain in G^- .

If O_l is assigned before O_{ij} on machine k in G^- , it can be started as early as $C^{E-}(PM_{ij}^k)$ and can be finished as late as S_{ij}^{L-} without delaying the required makespan $C_M(G)$ in G^- . Besides, O_l cannot violate the precedence relations of the same job. Thus, the assignable idle time interval for O_l can be defined as follows:

$$\max\{C^{E-}(PM_{ij}^k), C^{E-}(PJ_l)\} + p_{l,k} \leq \min\{S_{ij}^{L-}, S^{L-}(SJ_l)\} \quad (12)$$

The above moving process is repeated until all critical operations are moved. Let N_l be the number of positions to move O_l feasibly, then the total number of moving

$$\text{neighbors of a solution is } N_{\text{total}} = \sum_{l=1}^{N_c} N_l.$$

Moving critical operations will obtain new solutions. For the FJSP, there may be more than one critical path for a schedule. To improve a solution, all its critical paths should be changed. So, a solution with fewer critical paths is more likely to be improved. Thus, in our algorithm, the new solution will replace the old one if one of the following conditions is satisfied:

1. The new solution has a smaller makespan than the old solution.
2. The new solution with the same makespan as the old one has fewer critical paths.

To take advantage of the move of critical operations, we propose the following local search procedure based on moving operations in the onlooker bee phase, where the depth of local search is proportional to the number of onlooker bees N_i (Fig. 7).

4.7 The updating mechanism of population in the scout bee phase

The scout bees produce food sources randomly in the searching space to enhance the ability of exploration. In our ABC algorithm, the updating mechanism of population by generating scout bees is implemented as follows:

Step 1. Initialize scout bees with the method of the population initialization in Section 4.3 and update g_{best} if possible.

Step 2. Determine the worst n_{sb} employed bees and replace them with the scout bees if the scout bees are better.

With this process, the searching behavior of the whole ABC algorithm can be enriched, and the premature convergence of the algorithm can be avoided to some extent.

4.8 The framework of ABC for FJSP

Based on the above design, an ABC framework is proposed as illustrated in Fig. 8. It can be seen that the ABC algorithm not only applies employed bees and scout bees to perform exploration for promising solutions within the entire region but it also applies onlooker bees to perform exploitation for solution improvement. Due to the difficulty in solving the FJSP, the ABC algorithm uses a hybrid initialization scheme, multiple crossover operators for machine assignment and operation sequence, mutation for machine assignment, local search based on critical path, and population updating strategy. Since the exploration and exploitation are stressed and balanced, it is expected to achieve good performances for solving the FJSP.

5 Experimental results

To test the performance of the proposed ABC algorithm, numerical simulations are carried out with two well-studied benchmark sets including five Kacem instances [4] and ten BRdata problems [2]. The algorithm is implemented in C++ and run on 2.83-GHz PC with 3.21-GB RAM. Besides, we compare the ABC algorithm with several recently published algorithms. Same as literature, for each instance, we run the algorithm 50 times independently and summarize the statistical results.

5.1 Test on the Kacem instances

Considering the five Kacem instances [4] that range from four jobs five machines to 15 jobs ten machines, we test the performances of ABC algorithm and compare it with several algorithms including AL + CGA [5], GENACE [38], PSO + TS [37], PVNS [15], KBACO [14], and TSPCB [19]. For this set of instances, the parameters of the ABC algorithm are set as follows: the maximum number of generations $\text{gen}_{\text{max}} = n \times m$, the maximum number of no improvements $\text{ter}_{\text{max}} = 2 \times n$, the number of employed bees $n_{\text{eb}} = n$, the number of onlooker bees $n_{\text{ob}} = 4 \times n_{\text{eb}}$, and the number of scout bees $n_{\text{sb}} = n_{\text{eb}} \times 10\%$. The best results of all the algorithms are listed in Table 2, where C^* is the optimal values found so far. We also give the average results (AVG), the standard derivation (SD), and average CPU time (TAVG) of the ABC algorithm. The results of the comparative algorithms are directly from literatures, but most values such as AVG and SD are unavailable.

From Table 2, it can be seen that the ABC algorithm is the best among all the algorithms, and the ABC algorithm can consistently obtain the optimal values for all the five instances. According to the good performances in the best, average, derivation values, it is concluded that the ABC is of good searching quality and robustness. In addition, the computational time of the ABC algorithm is very short; for example, it can solve the 15 jobs ten machines optimally with about 5 s.

5.2 Test on the BRdata instances

Next we carry out simulation test and comparison with the ten BRdata instances [2] that range from ten jobs six machines to 20 jobs 15 machines. We compare our ABC algorithm with several existing algorithms, including TS [2], GA [24], LEGA [7], PVNS [15], KBACO [14], and TSPCB [19].

For this set of instances, the parameters of the ABC algorithm are set as follows: $\text{gen}_{\text{max}} = 5 \times n \times m$, ter_{max} equals to the total number of operations, $n_{\text{eb}} = 2 \times n$, $n_{\text{ob}} = 5 \times n_{\text{eb}}$, and $n_{\text{sb}} = n_{\text{eb}} \times 10\%$.

In Table 3, we list the best value (BEST), average value (AVG), and standard derivation (SD) of LEGA, PVNS, KBACO, TSPCB, and ABC. The results of the comparative algorithms are directly from literatures, where only the best values are available for TS and GA. For ABC, the average CPU time (TAVG) of 50 independent runs is also provided. Besides, the relative percentage improvement imp (percent) of ABC to other algorithms is listed Table 4. The relative percentage improvement is defined as follows:

$$\text{imp} = [(\text{MK}_c - \text{MK}_{\text{ABC}}) / \text{MK}_c] \times 100 \quad (13)$$

where MK_{ABC} and MK_c are the best makespan values obtained by ABC and the comparative algorithm, respectively.

From Table 3, it can be seen that our ABC performs better than other algorithms in solving almost all the instances in terms of the best results, average results, and standard derivation. In particular, as for the best results, ABC outperforms TS [2] in nine out of ten instances, outperforms both LEGA [7] and KBACO [14] in seven out of ten instances, outperforms TSPCB [19] in five out of ten instances, outperforms GA [24] in four out of ten instances, and outperforms PVNS [15] in two out of ten problems. So, it is concluded that the ABC algorithm has more powerful optimization capability in solving the FJSP. The same conclusion can be drawn from Table 4, since ABC improves the results of all other algorithms on average. Moreover, the SD values of ABC are relatively small, so it is concluded that ABC is robust on initial conditions. As for the efficiency, it can be seen that the average running time of ABC is acceptable, even for relatively larger-scale instances.

In Fig. 9, the Gantt chart for the best solution of the Mk7 obtained by ABC is illustrated. For this instance, only ABC and GA can obtain this solution. In addition, Fig. 10 depicts the convergence curves of the average makespan and the best makespan of the population when using ABC to solve MK9 instance in a typical run. To such an instance with 20 jobs ten machines, from Fig. 10, it can be seen that ABC obtains the best makespan with 44 generations of evolution and the whole population converges to the best solution with 50 generations of evolution. So, the ABC algorithm is very efficient in solving the FJSP.

According to the simulation tests and comparison study based on two sets of benchmarks, it is demonstrated that the proposed ABC algorithm is effective, efficient, and robust in solving the complex FJSP. The effectiveness owes to the stress of the balance of exploitation and exploration in designing the ABC algorithm, especially by using a hybrid initialization scheme, multiple crossover operators for machine assignment and operation sequence, mutation for machine assignment, local search based on critical path, and population updating strategy.

6 Conclusions

To the best of our knowledge, this was the first research work on the application of ABC algorithm to the FJSP with makespan criterion. Based on the special encoding and decoding schemes, effective searching operators including hybrid initialization, crossover, mutation, local search, and population updating were well designed in the employed bee phase, onlooker bee phase, and scout bee phase. Due to the balance of global exploration and local exploitation, the

proposed ABC algorithm was capable to solve the FJSP effectively, efficiently, and robustly, which was demonstrated by simulation tests and comparisons to several existing algorithms. The future work is to study the ABC algorithm for the multi-objective scheduling problems and stochastic scheduling problems by considering multi-objective handling technique and non-deterministic optimization techniques.

Acknowledgments This research is partially supported by National Science Foundation of China (61174189, 61025018, 70871065 and 60834004), Program for New Century Excellent Talents in University (NCET-10-0505), Doctoral Program Foundation of Institutions of Higher Education of China (20100002110014), the National Key Basic Research and Development Program of China (No.2009CB320602) and National Science and Technology Major Project of China (No.2011ZX02504-008).

References

1. Bruker P, Schlie R (1990) Job-shop scheduling with multi-purpose machines. *Computing* 45(4):369–375
2. Brandimarte P (1993) Routing and scheduling in a flexible job shop by tabu search. *Ann Oper Res* 41(3):157–183
3. Paulli J (1995) A hierarchical approach for the FMS scheduling problem. *Eur J Operat Res* 86(1):32–42
4. Kacem I, Hammadi S, Borne P (2002) Pareto-optimality approach for flexible job-shop scheduling problems: hybridization of evolutionary algorithms and fuzzy logic. *Math Comput Simul* 60(3–5):245–276
5. Kacem I, Hammadi S, Borne P (2002) Approach by localization and multi-objective evolutionary optimization for flexible job-shop scheduling problems. *IEEE Trans Sys Man Cybern Part C* 32(1):1–13
6. Xia WJ, Wu ZM (2005) An effective hybrid optimization approach for multi-objective flexible job-shop scheduling problems. *Comput Ind Eng* 48(2):409–425
7. Ho NB, Tay JC, Lai EMK (2007) An effective architecture for learning and evolving flexible job-shop schedules. *Eur J Operat Res* 179(2):316–333
8. Saidi-mehrabad M, Fattahi P (2007) Flexible job shop scheduling with tabu search algorithms. *Int J Adv Manuf Tech* 32(5–6):563–570
9. Fattahi P, Saidi M, Jolai F (2007) Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *J of Int Manuf* 18(3):331–342
10. Gao J, Sun LY, Gen M (2008) A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. *Com Oper Res* 35(9):2892–2907
11. Chen JC, Chen KH, Wu JJ, Chen CW (2008) A study of the flexible job shop scheduling problem with parallel machines and reentrant process. *Int J Adv Manuf Tech* 39(3–4):344–354
12. Zhang GH, Gao L, Li XY, Li PG (2008) Variable neighborhood genetic algorithm for the flexible job shop scheduling problems. *Lect Notes Com Sci* 5315:503–512
13. Gen M, Gao J, Lin L (2009) Multistage-based genetic algorithm for flexible job-shop scheduling problem. *Studies Com Int* 187:183–196
14. Xing LN, Chen YW, Wang P, Zhao QS, Xiong J (2010) A knowledge-based ant colony optimization for flexible job shop scheduling problems. *Applied Soft Com* 10(3):888–896
15. Yazdani M, Amiri M, Zandieh M (2010) Flexible job-shop scheduling with parallel variable neighborhood search algorithm. *Expert Sys Appl* 37(1):678–687

16. Moradi E, Fatemi Ghomi SMT, Zandieh M (2010) An efficient architecture for scheduling flexible job-shop with machine availability constraints. *Int J Adv Manuf Tech* 51(1–4):325–339
17. Defersha FM, Chen M (2010) A parallel genetic algorithm for a flexible job-shop scheduling problem with sequence dependent setups. *Int J Adv Manuf Tech* 49(1–4):263–279
18. Zhang GH, Gao L, Shi Y (2011) An effective genetic algorithm for the flexible job-shop scheduling problem. *Experts Sys Appl* 38(4):3563–3573
19. Li JQ, Pan QK, Suganthan PN, Chua TJ (2011) A hybrid tabu search algorithm with an efficient neighborhood structure for the flexible job shop scheduling problem. *Int J Adv Manuf Tech* 52(5–8):683–697
20. Dauzere-Peres S, Paulli J (1997) An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. *Ann Oper Res* 70:281–306
21. Mastrolilli M, Gambardella LM (2000) Effective neighborhood functions for the flexible job shop problem. *J Sched* 3(1):3–20
22. Baykasoglu A (2002) Linguistic-based meta-heuristic optimization model for flexible job shop scheduling. *Int J Product Res* 44(17):4523–4543
23. Chan FTS, Wong TC, Chan LY (2006) Flexible job-shop scheduling problem under resource constraints. *Int J Product Res* 44(11):2071–2089
24. Pezzella F, Morganti G, Ciaschetti G (2008) A genetic algorithm for the flexible job-shop scheduling problem. *Comput Oper Res* 35(10):3202–3212
25. Tay JC, Ho NB (2008) Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Comput Ind Eng* 54(3):453–473
26. Moslehi G, Mahnam M (2011) A Pareto approach to multi-objective flexible job-shop scheduling problem using particle swarm optimization and local search. *Int J Product Econ* 129(1):14–22
27. Karaboga D (2005) An idea based on honey bee swarm for numerical optimization, Technical Report TR06. Computer Engineering Department, Erciyes University, Turkey
28. Karaboga N (2009) A new design method based on artificial bee colony algorithm for digital IIR filters. *J Frankl Inst* 346(4):328–348
29. Karaboga D, Akay B (2009) A comparative study of artificial bee colony algorithm. *Appl Math Comput* 24(1):108–132
30. Karaboga D, Basturk B (2008) On the performance of artificial bee colony (ABC) algorithm. *Applied Soft Comput* 8(1):687–697
31. Karaboga D, Basturk B (2007) A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *J Global Optim* 39(3):459–471
32. Baykasoglu A, Ozbakir L, Tapkan P (2007) Artificial bee colony algorithm and its application to generalized assignment problem. In: *Swarm intelligence focus on ant and particle swarm optimization*. I-Tech Education and Publishing, Vienna, pp 113–144
33. Wong LP, Low MYH, Chong CS (2008) Bee colony optimization with local search for traveling salesman problem. In: *Proc. of 6th IEEE International Conference on Industrial Informatics*, pp 1019–1025
34. Pan QK, Tasgetiren MF, Suganthan PN, Chua TJ (2011) A discrete artificial bee colony algorithm for the lot-streaming flow shop scheduling problem. *Inf Sci* 181(12):2455–2468
35. Wang XJ, Gao L, Zhang CY, Shao XY (2010) A multi-objective genetic algorithm based on immune and entropy principle for flexible job-shop scheduling problem. *Int J Adv Manuf Tech* 51(5–8):757–767
36. Watanabe M, Ida K, Gen M (2005) A genetic algorithm with modified crossover operator and search area adaptation for the job-shop scheduling problem. *Comput Ind Eng* 48(4):743–752
37. Zhang GH, Shao XY, Li PG, Gao L (2009) An effective hybrid particle swarm optimization algorithm for multi-objective flexible job-shop scheduling problem. *Comput Ind Eng* 56(4):1309–1318
38. Tay JC, Wibowo D (2004) GENACE: an efficient cultural algorithm for solving the flexible job-shop problem. In: *Proc. of the IEEE Congress on Evolutionary Computation*, pp 1759–1766